

# Тяжелый селектор

В этой статье мы поговорим о производительности CSS-селекторов. Время, которое уходит на пересчёт стилей, зависит от многих вещей, но под контролем отдельных разработчиков обычно находится только количество элементов, для которых требуется произвести расчёты. Этот же показатель сильнее всего влияет на производительность, поэтому важно свести его к минимуму. Для этого вспомним главную вещь, влияющую на количество элементов.

## Селекторы применяются справа налево

Т.е. если мы возьмём селектор `.foo .bar {}`, то сначала браузер найдёт на странице все элементы `.bar`, потом среди них найдёт те, в пределах которых есть элемент `.foo`, и только тогда применит правило. Именно поэтому использование селекторов по типу элемента (например, `div {}`) не рекомендуется.

Теперь сравним между собой несколько селекторов и заодно научимся искать медленные.

[См. демо-пример](#)

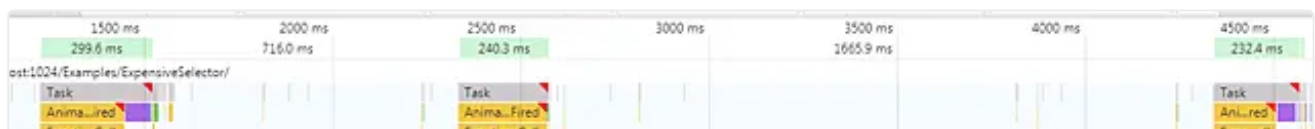
На этой странице подключается CSS со следующими правилами:

▼ Пример кода

```
1  .DebuggingExamples-ExpensiveSelector_block div {
2      font-size: 14px;
3  }
4  .DebuggingExamples-ExpensiveSelector_class
5  .DebuggingExamples-ExpensiveSelector__testDiv {
6      font-size: 15px;
7  }
8  .DebuggingExamples-ExpensiveSelector_anotherClass
9  .DebuggingExamples-ExpensiveSelector__testDiv {
10     font-size: 16px;
11 }
```

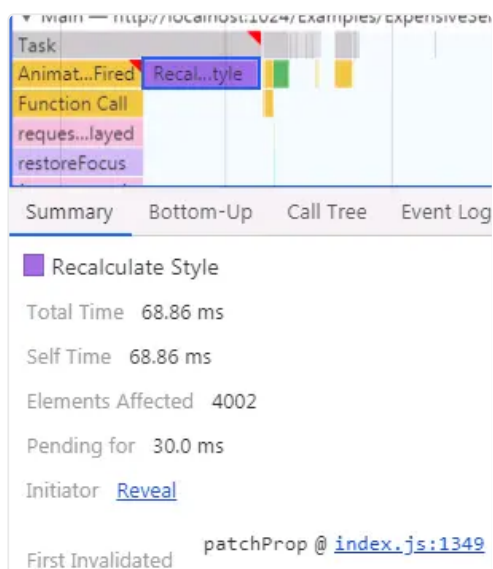
`.DebuggingExamples-ExpensiveSelector_block`, `.DebuggingExamples-ExpensiveSelector_class` и `.DebuggingExamples-ExpensiveSelector_anotherClass` навешиваются на корень примера, а в самом примере находится список со случайными данными и одна строка текста над списком. Всё сверстано `div`'ами, на строчке висит класс `.DebuggingExamples-ExpensiveSelector__testDiv` и все эти три селектора затрагивают только эту строчку. Кнопка "Обновить стили" переключается между классами, начиная с `.DebuggingExamples-ExpensiveSelector_block`.

Если мы запустим профилировщик и три раза нажмём на кнопку, то получим следующую картину:

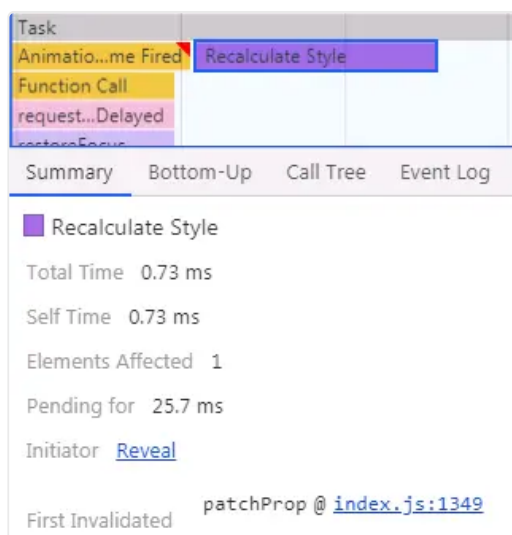


Нас волнуют фиолетовые участки.

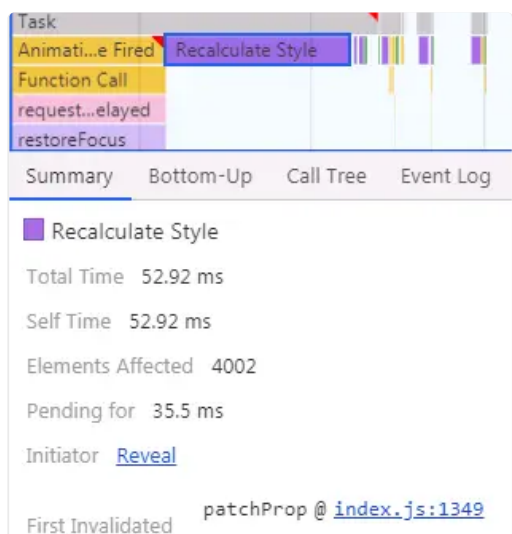
Когда мы переключаемся с `.DebuggingExamples-ExpensiveSelector_block` на `.DebuggingExamples-ExpensiveSelector_class`, то затронутыми оказываются 4002 элемента и это занимает 68.86ms. Довольно долго, если учесть, что на самом деле изменится только один элемент.



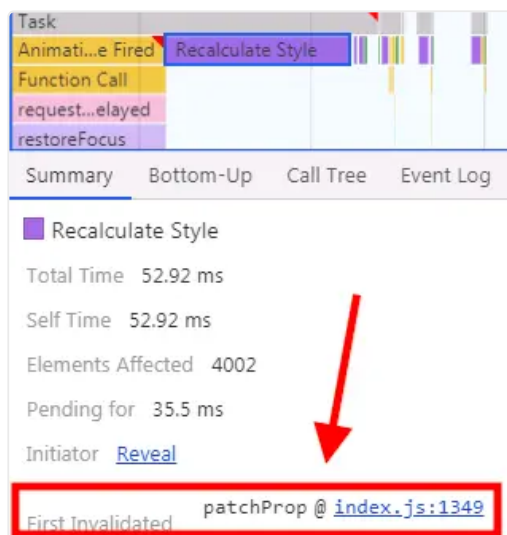
Во втором случае ситуация уже гораздо лучше: 0.73ms и всего один элемент. Почему так? Потому что ни второй, ни третий селектор никак не воздействуют на div в списке, а значит не затрагивают большую часть элементов на странице.



Наконец, после третьего нажатия мы вернёмся к исходному селектору и ожидаемо получим большой пересчёт стилей: 52.92ms и 4002 элемента.



Допустим, мы бы не знали какой класс переключается и у нас был бы только профиль, в котором был тяжёлый пересчёт стилей: что делать в таком случае? Здесь нужно обратить внимание в самый низ описания Recalculate Style, на строчку First Invalidated:



Там будет указана причина пересчётов. В большинстве случаев там будет ссылка на JS функцию, в которую можно поставить точку останова. Там уже можно будет посмотреть что меняется, старое значение и новое значение.



Дальше остаётся только смотреть на css и если селектор выглядит подозрительно, то пробовать повторить действие без него. Если проблема уходит, то нужно думать на тему оптимизации селектора. В большинстве случаев это означает, что на ваш элемент нужно навесить уникальный класс и пользоваться им.